

Le QCM Python et pandas

Ce que tu écris vraiment /20



Avant un test technique ou une prise de poste Data Analyst. Durée conseillée : 25 min · 4 paliers · corrigé détaillé.

En code review, je vois souvent le même schéma : un notebook qui tourne, recopié depuis quelques mois, sans que la personne sache pourquoi elle appelle `reset_index` ou pourquoi son `groupby` perd des lignes. Ce test isole exactement ces blocs, pas ta capacité à reconnaître une syntaxe déjà vue.

Repères : 12/20 = tu recopies encore sans comprendre, 17/20 = profil qu'on garde pour un test technique.

Candidat _____ Date _____ Note _____ /20

Palier 1 : lire et nettoyer un fichier sale.

La première chose qu'un recruteur regarde en revue de code : est-ce que tu inspectes vraiment un fichier avant d'écrire la première ligne de traitement, ou est-ce que tu enchaînes un pipeline standard sans vérifier.

Question 1. Tu charges un CSV avec `pd.read_csv('ventes.csv')` sans `parse_dates`. La colonne `date_commande` contient des valeurs comme `'2024-03-15'`. Après le chargement, quel est le dtype de cette colonne ?

- a) `datetime64[ns]`, pandas détecte automatiquement les dates au format ISO standard.
- b) `object` (ou `str` en pandas 3.x).
- c) `int64`, pandas convertit les dates en timestamp Unix.
- d) `category`, pandas optimise automatiquement les colonnes répétitives.

Question 2. Un `DataFrame` reçu d'une API contient une colonne `email` avec `'contact@x.fr'`, la chaîne `'NA'`, une chaîne vide `''` et une vraie cellule `None`. Tu lances `df['email'].isna()`. Que renvoie exactement ce test ?

- a) `True` uniquement pour la cellule `None`; `'NA'` et `''` restent détectées comme `False`.
- b) `True` pour les trois cas, pandas reconnaissant `'NA'` et `''` comme manquants par défaut.
- c) `False` pour tous les cas, `isna()` ne s'appliquant pas à du texte.
- d) Une erreur, `isna()` ne fonctionnant pas sur une colonne `object`.

Question 3. Un fichier client dupliqué contient deux lignes pour le même `client_id`, mais avec un email légèrement différent (espace en trop). Tu veux garder une seule ligne par client. Quelle instruction correspond à ce besoin ?

- a) `df.drop_duplicates()` sans argument, qui compare toutes les colonnes.
- b) `df.drop_duplicates(subset='email')`.
- c) `df.drop_duplicates(subset='client_id')`.
- d) `df.groupby('client_id').first()`, qui agrège les emails des doublons en les concaténant.

Question 4. Une colonne `ville` contient des valeurs mal formatées comme `' Paris '`, `'LYON'`, `'nantes '`. Tu écris `df['ville'] = df['ville'].str.strip().str.lower()`. Que se passe-t-il ?

- a) Une erreur : impossible d'enchaîner deux méthodes `.str` sur la même ligne.
- b) Chaque valeur est nettoyée des espaces puis mise en minuscule, dans l'ordre d'écriture.
- c) Seule `.str.lower()` s'applique, `.str.strip()` étant ignorée.
- d) L'enchaînement fonctionne mais réinitialise l'index à 0, 1, 2, comme après un tri complet.

Question 5. Une colonne `quantite` contient '10', '25', 'inconnu'. Tu veux la convertir en nombre sans que le script plante sur la valeur invalide. Quelle instruction correspond à ce besoin ?

- a) `df['quantite'].astype(int)`, qui convertit tout sauf les valeurs invalides.
- b) `df['quantite'].astype(float)`, qui remplace automatiquement 'inconnu' par 0.
- c) `pd.to_numeric(df['quantite'], errors='raise')`, qui ignore les erreurs.
- d) `pd.to_numeric(df['quantite'], errors='coerce')`, qui remplace 'inconnu' par NaN.

Palier 2 : sélectionner sans se faire piéger par l'index.

Cinq questions sur `loc`, `iloc`, le filtrage booléen et le chained assignment : l'endroit où un notebook qui tourne peut quand même écrire au mauvais endroit.

Question 6. Sur un DataFrame indexé de 0 à 9, tu écris `df.loc[2:5]` puis `df.iloc[2:5]`. Combien de lignes chaque instruction renvoie-t-elle ?

- a) `loc[2:5]` renvoie 4 lignes, `iloc[2:5]` renvoie 3 lignes.
- b) Les deux renvoient exactement 3 lignes, comme un slice Python classique.
- c) `loc[2:5]` renvoie 3 lignes et `iloc[2:5]` renvoie 4 lignes.
- d) Les deux renvoient 4 lignes, pandas incluant toujours la borne finale.

Question 7. Tu écris `df[df['age'] > 18 & df['ville'] == 'Paris']` pour filtrer les clients majeurs à Paris. Que se passe-t-il à l'exécution ?

- a) Le filtre fonctionne normalement : pandas applique d'abord les comparaisons puis le ET logique.
- b) Python lève une `TypeError` : `&` est évalué avant les comparaisons.
- c) Le filtre s'exécute mais renvoie un DataFrame vide silencieusement.
- d) pandas corrige automatiquement la priorité des opérateurs et retourne le bon résultat.

Question 8. Dans un notebook, un collègue écrit `df[df['ville']=='Paris']['montant'] = 0` puis ignore l'avertissement affiché en rouge. Pourquoi cet avertissement ne doit pas être ignoré ?

- a) Un simple message de dépréciation sans conséquence sur les données.
- b) pandas refuse d'exécuter la ligne, donc rien ne peut être faux.
- c) L'avertissement ne concerne que les DataFrame de plus d'un million de lignes, en dessous de ce seuil aucune copie n'est créée.
- d) L'écriture porte sur une sélection intermédiaire, peut-être une copie : la modification risque de ne pas se propager.

Question 9. Après avoir filtré un DataFrame avec `df[df['montant']>100]`, l'index affiche 3, 7, 12... Tu veux un index propre de 0 à n, sans créer de colonne supplémentaire. Quelle instruction utiliser ?

- a) `df.set_index('montant')`.
- b) `df.reset_index(drop=True)`.
- c) `df.reset_index()` sans argument.
- d) `df.sort_index()`.

Question 10. Un script écrit `result = df['montant'].sum()`. Un collègue le remplace par `result = df[['montant']].sum()`. Quelle différence explique un comportement inattendu en aval ?

- a) `df['montant']` renvoie une Series ; `df[['montant']]` renvoie un DataFrame à une colonne.
- b) Les deux syntaxes renvoient le même résultat après un `.sum()`, pandas convertissant l'un vers l'autre automatiquement.
- c) `df[['montant']]` déclenche systématiquement une erreur de syntaxe.
- d) `df['montant']` renvoie un DataFrame, `df[['montant']]` renvoie une Series.

Palier 3 : agréger sans mentir sur les chiffres.

Cinq questions sur le coeur du poste d'analyste : `groupby`, `agg`, `transform`, `pivot_table`, et ce qui se joue quand un chiffre remonte en réunion sans que personne n'ait vérifié d'où il sort.

Question 11. Tu veux, pour chaque région, la somme du `montant` et la moyenne de la `quantite` en un seul appel. Quelle instruction fait exactement ça ?

- a) `df.groupby('region').agg({'montant': 'sum', 'quantite': 'mean'})`.
- b) `df.groupby('region').sum()`, qui applique la même agrégation à toutes les colonnes numériques.
- c) `df.groupby('region')[['montant', 'quantite']].sum()`.
- d) `df.groupby('region').mean()`, qui calcule la moyenne pour toutes les colonnes, montant compris.

Question 12. Tu veux ajouter une colonne `montant_moyen_region` qui répète, sur chaque ligne, la moyenne du montant de sa région, en gardant le nombre de lignes d'origine. Quelle méthode utiliser ?

- a) `df.groupby('region')['montant'].apply(lambda s: s.mean())`, qui conserve la forme d'origine car `apply` hérite automatiquement de l'index du `groupby`.
- b) `df.groupby('region')['montant'].transform('mean')`, aligné sur l'index d'origine.
- c) `df.groupby('region')['montant'].sum()`, puis un `merge` manuel obligatoire.
- d) `df.groupby('region')['montant'].agg('mean')`, qui garde le même nombre de lignes que `df`.

Question 13. Tu écris `df.pivot_table(index='region', values='ventes')` sans préciser `aggfunc`. Quelle valeur obtiens-tu pour chaque région ?

- a) La moyenne (`mean`) des ventes de la région.
- b) La somme (`sum`) des ventes de la région : `pivot_table` agrège par addition quand rien n'est précisé.
- c) Le nombre de lignes (`count`) de la région.
- d) Une erreur : `aggfunc` doit être précisé pour que pandas sache quoi calculer.

Question 14. Tu veux le top 3 des clients par montant, pour chaque région, dans un rapport. Un simple `df.sort_values('montant', ascending=False).head(3)` ne convient pas. Pourquoi ?

- a) `sort_values` ne fonctionne pas avec des montants décimaux, il faut convertir en entier avant.
- b) Il faut utiliser `nlargest(3, 'montant')` sur tout le `DataFrame`, sans notion de région.
- c) Le problème vient de `head(3)`, qu'il faudrait remplacer par `head(-3)` pour prendre les derniers.
- d) `sort_values().head(3)` prend le top 3 global, toutes régions confondues.

Question 15. La colonne `region` contient quelques valeurs manquantes sur des commandes mal renseignées. Tu écris `df.groupby('region')['ventes'].sum()` pour ton rapport mensuel. Que deviennent les lignes avec `region` manquante ?

- a) Elles sont regroupées ensemble sous une clé `NaN` affichée dans le résultat, comme n'importe quelle autre valeur de groupe.
- b) pandas lève une erreur, `groupby` refusant les clés manquantes.
- c) Elles sont exclues du résultat par défaut.
- d) pandas les remplace automatiquement par `'Inconnu'` avant de grouper.

Palier 4 : les pièges qui trahissent un junior en test technique.

Cinq questions sur ce qui plombe une note à l'oral quand le code tourne sans planter, mais que le chiffre en sortie est un peu faux : fusions, typage, performance et valeurs manquantes.

Question 16. Tu fais `gauche.merge(droite, on='client_id', how='left')` pour enrichir tes commandes avec les infos client. La table `droite` contient deux lignes pour le même `client_id` (erreur de saisie). Quel est l'effet sur le résultat ?

- a) pandas déduplique automatiquement la table `droite` avant la fusion.
- b) Chaque commande de ce client apparaît en double dans le résultat : le nombre de lignes augmente.
- c) La fusion échoue avec une erreur de clé dupliquée.
- d) Seule la première ligne correspondante de `droite` est utilisée, les suivantes étant ignorées comme pour un `drop_duplicates` implicite.

Question 17. Une colonne `quantite` est en `int64`. Après une fusion, certaines lignes n'ont pas de correspondance et reçoivent `NaN`. Que devient le `dtype` de la colonne ?

- a) Elle reste en `int64`, `NaN` étant représenté comme 0.
- b) pandas lève une erreur de type au moment de la fusion.
- c) Elle passe automatiquement en `float64`.
- d) Elle passe en `object`, pandas mélangeant alors entiers et `NaN` comme pour toute colonne texte.

Question 18. Sur un `DataFrame` de 500 000 lignes, tu veux calculer `marge = prix - cout`. Un collègue écrit `df.apply(lambda row: row['prix'] - row['cout'], axis=1)`. Quel est le problème ?

- a) `apply(axis=1)` déclenche une erreur de mémoire au-delà de 100 000 lignes.
- b) Le résultat sera faux, `apply` ne pouvant pas soustraire deux colonnes.
- c) `apply(axis=1)` modifie le `DataFrame` original par erreur, sans faire de copie.
- d) `apply(axis=1)` traite les lignes une par une en Python pur, plus lent que l'écriture vectorisée.

Question 19. Pour isoler les commandes sans date de livraison, un collègue écrit `df[df['date_livraison'] == np.nan]`. Quel est le résultat ?

- a) Le filtre isole correctement toutes les lignes avec une date manquante.
- b) pandas lève une erreur car on ne peut pas comparer une colonne à `np.nan`.
- c) Le filtre renvoie l'inverse : toutes les lignes sauf celles avec une date manquante.
- d) Le filtre renvoie toujours un `DataFrame` vide, une comparaison à `NaN` valant toujours `False`.

Question 20. La colonne `type_contrat` ne contient que 4 valeurs possibles (`CDI`, `CDD`, `Stage`, `Alternance`) répétées sur 200 000 lignes. Quel est l'intérêt de `df['type_contrat'] = df['type_contrat'].astype('category')` ?

- a) Cela convertit automatiquement les valeurs en chiffres visibles dans les exports Excel.
- b) Cela réduit fortement l'empreinte mémoire de la colonne et peut accélérer tri et `groupby`.
- c) Cela empêche définitivement d'ajouter une cinquième valeur au `type_contrat`.
- d) Cela n'a aucun effet mesurable; `category` ne fait que renommer le `dtype` affiché par `dtypes`.

Corrigé, paliers 1 à 4.

Un point par bonne réponse. Compte ton score, puis relis surtout celles que tu ne peux pas expliquer à voix haute sans relancer un notebook.

- 1. b) Sans `parse_dates`, pandas lit une colonne date comme du texte : `dtype object` en pandas 2.x, `str` en pandas 3.x (PDEP-14). Le piège junior : filtrer ou trier cette colonne comme une date sans jamais l'avoir convertie.
- 2. a) `isna()` ne détecte que le `None/NaN` natif : `'NA'` et `' '` restent des chaînes tant qu'elles ne sont pas converties. Nuance : `read_csv` les convertit lui-même via sa liste `na_values` par défaut, ce piège vise les données déjà en mémoire. Le piège junior : croire qu'un `isna().sum()` à zéro veut dire un fichier propre.
- 3. c) `subset='client_id'` cible la clé métier; sans `subset`, `drop_duplicates()` compare toutes les colonnes et laisse passer les doublons dont l'email diffère d'un espace. Le piège junior : compter des clients uniques sans jamais préciser la clé.
- 4. b) `.str.strip().str.lower()` s'enchaîne normalement sur une `Series` texte et conserve l'index d'origine, sans réinitialisation. Le piège junior : penser qu'un enchaînement de méthodes `.str` casse la colonne ou l'index.
- 5. d) `astype(int)` lève une erreur sur `'inconnu'` et arrête le script; `pd.to_numeric(errors='coerce')` renvoie `NaN` à la place et laisse le traitement continuer. Le piège junior : utiliser `astype` par réflexe sur une colonne texte pas encore validée.

- 6. a)** `loc` inclut la borne finale du slice, `iloc` l'exclut comme un index Python classique : deux comportements différents sur une même écriture `[2:5]`. Le piège junior : copier un slice `loc` en `iloc` en pensant garder le même résultat.
- 7. b)** En Python, `&` est évalué avant `>` et `==` : l'expression calcule d'abord `18 & df['ville']`, ce qui lève une `TypeError` entre un entier et une colonne texte. Il faut parenthéser chaque condition, `(df['age']>18) & (df['ville']=='Paris')`. Le piège junior : croire que la syntaxe pandas suit l'ordre logique naturel d'une phrase.
- 8. d)** Écrire dans une sélection chaînée comme `df[cond]['col']` porte sur un résultat intermédiaire qui peut être une copie : l'affectation risque de ne pas modifier le DataFrame d'origine, sans plantage visible. Le piège junior : ignorer l'avertissement rouge parce que le script continue de tourner.
- 9. b)** `reset_index(drop=True)` renumérote l'index de 0 à n sans créer de colonne; `set_index` change la colonne utilisée comme index, ce qui répond à un besoin différent. Le piège junior : confondre les deux et perdre une colonne utile par erreur.
- 10. a)** `df['col']` renvoie une Series (un scalaire après `.sum()`), `df[['col']]` renvoie un DataFrame à une colonne (une Series indexée par nom après `.sum()`). Le piège junior : un script en aval qui attend un nombre et reçoit une structure indexée.
- 11. a)** `agg({'montant':'sum','quantite':'mean'})` applique une fonction différente par colonne en un seul appel, contrairement à `.sum()` ou `.mean()` globaux. Le piège junior : enchaîner deux `groupby` séparés puis un `merge` fastidieux pour le même résultat.
- 12. b)** `transform` renvoie une valeur par ligne alignée sur l'index d'origine, donc le même nombre de lignes que `df`; `apply` peut réduire la forme selon la fonction passée. Le piège junior : utiliser `apply` puis devoir refaire un `merge` pour rattacher le résultat.
- 13. a)** `pivot_table` utilise `mean` comme `aggfunc` par défaut quand rien n'est précisé, ce qui peut donner une moyenne là où un total était attendu. Le piège junior : lire un tableau croisé sans vérifier l'`aggfunc` et présenter une moyenne comme un total en réunion.
- 14. d)** `sort_values().head(3)` trie tout le DataFrame puis coupe globalement : le top 3 obtenu mélange les régions au lieu d'en donner un par groupe. Il faut grouper par région avant de trier, par exemple avec `groupby('region').apply(...)` ou `nlargest` par groupe. Le piège junior : livrer un classement où une seule région domine tout le rapport.
- 15. c)** Par défaut, `groupby` exclut les clés `NaN` (`dropna=True` par défaut) : les lignes correspondantes disparaissent du résultat sans message d'erreur. Le piège junior : présenter un total de rapport inférieur au total réel sans s'en rendre compte.
- 16. b)** Une clé dupliquée côté droit fait correspondre chaque ligne de gauche à plusieurs lignes de droite : le nombre de lignes du résultat augmente, sans erreur visible. Le piège junior : un total qui gonfle après une fusion, attribué à tort à une erreur de saisie ailleurs.
- 17. c)** `NaN` n'existe pas en `int64` : dès qu'une valeur manquante apparaît dans une colonne d'entiers, pandas la caste automatiquement en `float64`. Le piège junior : un test `== 5` qui échoue silencieusement après une fusion à cause de ce changement de type.
- 18. d)** `apply(axis=1)` exécute la fonction ligne par ligne en Python pur, alors que `df['prix'] - df['cout']` s'exécute en vectorisé (C sous le capot) : l'écart de vitesse devient net dès quelques centaines de milliers de lignes. Le piège junior : livrer un script qui tourne largement plus longtemps que nécessaire (souvent un ordre de grandeur, sans mesure prise) sans s'en apercevoir en test.
- 19. d)** Une comparaison `== np.nan` vaut toujours `False`, même sur une vraie valeur manquante : comportement standard hérité d'IEEE 754. Seule `isna()` détecte correctement les valeurs manquantes. Le piège junior : livrer un filtre qui renvoie zéro ligne sans lever d'erreur, donc sans alerter personne.
- 20. b)** `category` stocke des codes entiers en interne au lieu de répéter le texte, ce qui réduit fortement la mémoire et peut accélérer tri et `groupby` sur une colonne à faible cardinalité. Le piège junior : garder tout en `object` sur un fichier volumineux et découvrir le ralentissement en production.

Fait main, en solo.

Si ce test t'a aidé à repérer tes blocs à revoir, partage-le : il peut débloquent quelqu'un d'autre en pleine recherche de poste. Et si tu veux que je traite un sujet pandas précis, dis-le moi en commentaire : c'est là que je pioche mes prochains guides.

Dylan

